BÀI THỰC HÀNH

LAB 01

Cơ bản về Digital Output

Điều khiển GPIO với EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Mục lục

1 Giới thiệu . . . 2
1.1 Tổng quan bài lab . . . 2
1.2 Mục tiêu . . . 2
2 Kiến thức nền . . . 2
2.1 Digital Output và mức logic . . . 2
2.2 Digital Output và LED active-low . . . 2
2.3 Chu kỳ nháy LED . . . 3
3 Thiết lập phần cứng . . . 3
3.1 Phần cứng sử dụng . . . 3
3.2 Ánh xạ LED tích hợp . . . 4
4 EduFramework API . . . 4
4.1 `pinMode()` . . . 4
4.2 `digitalWrite()` . . . 5
4.3 `delay()` . . . 5
5 Bài thực hành . . . 5
5.1 Yêu cầu . . . 6
5.2 Chương trình . . . 6
5.3 Kiểm chứng . . . 6
6 Mở rộng . . . 6
6.1 Đảo trạng thái với `digitalToggle()` . . . 7
6.2 Định thời với `millis()` . . . 7
6.3 Bài tập mở rộng . . . 7
7 Tài liệu tham khảo . . . 8

1 Giới thiệu

1.1 Tổng quan bài lab

Digital Input/Output (Digital I/O) là một chức năng cơ bản của GPIO, cho phép vi điều khiển trao đổi các trạng thái logic với phần cứng bên ngoài. Khi một chân GPIO được cấu hình làm đầu ra, chương trình có thể thiết lập trạng thái logic của chân để tạo tín hiệu số [1] .

Bài lab này được thiết kế để giới thiệu Digital Output thông qua các API của EduFramework. LED tích hợp trên MaaZEDU Development Board được sử dụng làm đầu ra trực quan. Bằng cách luân phiên hai mức logic `HIGH` và `LOW` theo thời gian, chương trình tạo thành ứng dụng nháy LED cơ bản.

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Giải thích nguyên lý cơ bản của Digital Output và ý nghĩa của hai mức logic `HIGH` , `LOW` .
2. Xác định mối quan hệ giữa mức logic của GPIO và trạng thái của LED active-low.
3. Sử dụng các API `pinMode()` , `digitalWrite()` và `delay()` để điều khiển Digital Output.
4. Xây dựng và kiểm chứng ứng dụng nháy LED trên MaaZEDU Development Board.

2 Kiến thức nền

2.1 Digital Output và mức logic

GPIO (General-Purpose Input/Output) là các chân có thể được cấu hình để thực hiện chức năng vào hoặc ra số. Khi được cấu hình ở chế độ Digital Output, trạng thái logic tại chân được điều khiển bởi chương trình. Trong EduFramework, chế độ đầu ra được biểu diễn bằng `OUTPUT` , trong khi hai trạng thái logic được biểu diễn bằng `HIGH` và `LOW` .

Trạng thái	Giá trị	Ý nghĩa
`LOW`	`0U`	Mức logic thấp.
`HIGH`	`1U`	Mức logic cao.

`HIGH` và `LOW` biểu diễn trạng thái logic, không phải một giá trị điện áp cố định. Điện áp thực tế và các giới hạn điện của chân I/O phụ thuộc vào đặc tính của vi điều khiển. Đối với S32K1xx, các thông số này được quy định trong Data Sheet của NXP [3] .

2.2 Digital Output và LED active-low

LED thường được sử dụng để biểu diễn trực quan trạng thái của một đầu ra số. Tuy nhiên, quan hệ giữa mức logic của GPIO và trạng thái sáng hoặc tắt của LED phụ thuộc vào cách LED được kết nối trong mạch.

Cấu hình	LED bật	LED tắt
Active-high	`HIGH`	`LOW`
Active-low	`LOW`	`HIGH`

Các LED tích hợp được sử dụng trong cấu hình phần cứng của bài lab hoạt động theo nguyên lý active-low. Vì vậy, mức `LOW` làm LED sáng, trong khi mức `HIGH` làm LED tắt.

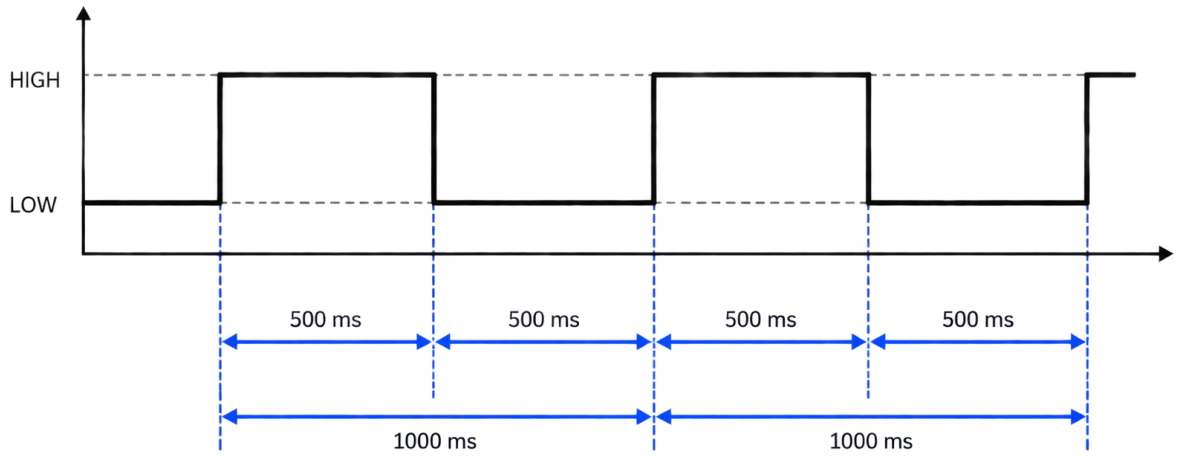
Ghi chú

`HIGH` và `LOW` mô tả trạng thái logic của GPIO; bật và tắt mô tả trạng thái của thiết bị được kết nối. Quan hệ giữa hai trạng thái này được quyết định bởi cấu hình phần cứng.

2.3 Chu kỳ nháy LED

Ứng dụng nháy LED thay đổi tuần hoàn trạng thái của LED. Trong bài lab này, LED được giữ ở trạng thái sáng trong `500 ms` và trạng thái tắt trong `500 ms`. Do đó, thời gian của một chu kỳ hoàn chỉnh là:

$$T = 500 \ \text{ms} + 500 \ \text{ms} = 1000 \ \text{ms} = 1 \ \text{s}$$



Hình 1.2.1. Quan hệ thời gian của ứng dụng nháy LED

Khoảng thời gian giữa các lần thay đổi trạng thái được tạo bằng `delay()`, giúp hai trạng thái của LED có thể được quan sát trực tiếp trên phần cứng.

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Bài lab sử dụng trực tiếp LED tích hợp trên MaaZEDU Development Board và không yêu cầu đấu nối thêm linh kiện bên ngoài.

Phần cứng	Mô tả
MaaZEDU Development Board	Board phát triển sử dụng vi điều khiển S32K144.
USB Cable	Kết nối board với máy tính để cấp nguồn và nạp chương trình.

Bảng 1.3.1. Phần cứng sử dụng trong bài thực hành

3.2 Ánh xạ LED tích hợp

EduFramework cung cấp các Logical Pin để truy cập LED tích hợp trong mã nguồn ứng dụng. Ánh xạ phần cứng được sử dụng trong framework như sau:

Thành phần	Logical Pin	MCU Pin	Chức năng
LED đỏ	`LED_RED`	`PTD15`	Digital Output
LED xanh dương	`LED_BLUE`	`PTD16`	Digital Output
LED xanh lá	`LED_GREEN`	`PTD0`	Digital Output

Bảng 1.3.2. Ánh xạ các LED tích hợp trên MaaZEDU Development Board

Bài thực hành chính sử dụng `LED_RED`, được EduFramework ánh xạ tới chân `PTD15` của S32K144.

4 EduFramework API

Ứng dụng nháy LED sử dụng ba API: `pinMode()` để cấu hình Digital Output, `digitalWrite()` để thiết lập mức logic và `delay()` để tạo khoảng thời gian giữa các lần thay đổi trạng thái. Cách tổ chức các API này được xây dựng theo mô hình lập trình quen thuộc của Arduino [2].

4.1 `pinMode()`

`pinMode()` được sử dụng để cấu hình chế độ hoạt động của một chân digital.

`pinMode`

Cấu hình chế độ hoạt động cho Logical Pin được chỉ định.

Cú pháp

```
pinMode(pin, mode);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
`pin`	Logical Pin	Chân cần cấu hình, ví dụ `LED_RED`.
`mode`	`OUTPUT`	Cấu hình chân làm Digital Output.

Giá trị trả về

Không trả về giá trị.

Ví dụ:

```
pinMode(LED_RED, OUTPUT);
```

4.2 `digitalWrite()`

`digitalWrite()` được sử dụng để thiết lập mức logic của một chân Digital Output.

`digitalWrite`

Thiết lập mức logic cho Logical Pin được chỉ định.

Cú pháp

```
digitalWrite(pin, value);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
`pin`	Logical Pin	Chân cần điều khiển, ví dụ `LED_RED`.
`value`	`HIGH` / `LOW`	Mức logic cần thiết lập cho chân output.

Giá trị trả về

Không trả về giá trị.

Ví dụ:

```
digitalWrite(LED_RED, LOW);
digitalWrite(LED_RED, HIGH);
```

4.3 `delay()`

`delay()` tạo khoảng thời gian chờ trước khi chương trình tiếp tục thực hiện lệnh tiếp theo.

`delay`

Tạo khoảng thời gian chờ theo đơn vị millisecond.

Cú pháp

```
delay(ms);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
`ms`	Thời gian	Khoảng thời gian cần chờ, tính bằng millisecond.

Giá trị trả về

Không trả về giá trị.

Ví dụ:

```
delay(500U);
```

Lệnh trên tạo khoảng thời gian chờ `500 ms`.

5 Bài thực hành

Tạo một project PlatformIO sử dụng EduFramework cho MaaZEDU Development Board.

5.1 Yêu cầu

Xây dựng chương trình điều khiển LED đỏ tích hợp nhấp nháy liên tục. LED sáng trong `500 ms`, sau đó tắt trong `500 ms` và quá trình được lặp lại liên tục.

5.2 Chương trình

Trong `src/main.c`, triển khai chương trình như sau:

```
#include "Arduino.h"

int main(void)
{
    setup();

    pinMode(LED_RED, OUTPUT);

    while (1)
    {
        digitalWrite(LED_RED, LOW);
        delay(500U);

        digitalWrite(LED_RED, HIGH);
        delay(500U);
    }

    return 0;
}
```

Mã nguồn 1.5.1. Chương trình nháy LED sử dụng EduFramework

`setup()` khởi tạo EduFramework trước khi các API được sử dụng. Sau đó, `pinMode()` cấu hình `LED_RED` thành Digital Output. Trong vòng lặp `while (1)`, `digitalWrite()` lần lượt thiết lập `LOW` và `HIGH`; mỗi trạng thái được duy trì trong `500 ms` bằng `delay()`.
Do LED hoạt động theo cấu hình active-low, `LOW` tương ứng với trạng thái sáng và `HIGH` tương ứng với trạng thái tắt.

5.3 Kiểm chứng

Build và nạp chương trình xuống MaaZEDU Development Board, sau đó quan sát LED đỏ tích hợp.

KẾT QUẢ MONG ĐỢI

LED đỏ sáng khoảng `500 ms`, tắt khoảng `500 ms` và lặp lại liên tục. Một chu kỳ nhấp nháy hoàn chỉnh kéo dài khoảng `1 s`.

6 Mở rộng

Ngoài các API cơ bản được sử dụng trong bài thực hành, EduFramework còn cung cấp các API bổ sung và nâng cao phục vụ nhiều kiểu bài toán khác nhau. Trong phần mở rộng này, `digitalToggle()` được sử dụng để thực hiện thao tác đảo trạng thái Digital Output, trong khi `millis()` được sử dụng để theo dõi thời gian thực thi mà không cần tạo khoảng chờ bằng `delay()`.

6.1 Đảo trạng thái với `digitalToggle()`

`digitalToggle()` đảo mức logic hiện tại của một Digital Output. Nếu trạng thái hiện tại là `LOW`, chân được chuyển sang `HIGH`; ngược lại, nếu trạng thái hiện tại là `HIGH`, chân được chuyển sang `LOW`.

`digitalToggle`

Đảo trạng thái logic hiện tại của Logical Pin được chỉ định.

Cú pháp

```
digitalToggle(pin);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
`pin`	Logical Pin	Chân Digital Output cần đảo trạng thái.

Giá trị trả về

Không trả về giá trị.

Mở rộng: Viết lại chương trình nháy LED bằng `digitalToggle()` sao cho trạng thái LED vẫn được thay đổi sau mỗi `500 ms`.

6.2 Định thời với `millis()`

`delay()` tạo một khoảng chờ và giữ luồng thực thi cho đến khi khoảng thời gian yêu cầu kết thúc. Trong các ứng dụng cần tiếp tục xử lý những tác vụ khác trong khi theo dõi thời gian, `millis()` có thể được sử dụng để xác định thời gian đã trôi qua mà không cần dừng vòng lặp chính.

`millis`

Đọc số millisecond đã trôi qua kể từ khi hệ thống thời gian được khởi tạo.

Cú pháp

```
millis();
```

Giá trị trả về

Thời gian đã trôi qua, tính bằng millisecond.

Một khoảng thời gian có thể được kiểm tra theo nguyên tắc:

```
if ((millis() - previousTime) >= interval)
{
    /* Periodic operation */
}
```

Mã nguồn 1.6.1. Kiểm tra khoảng thời gian bằng `millis()`

Trong đó, `previousTime` lưu mốc thời gian của lần xử lý trước và `interval` xác định khoảng thời gian giữa hai lần xử lý.

Mở rộng: Viết lại ứng dụng nháy LED với khoảng đảo trạng thái `500 ms` mà không sử dụng `delay()`. Khi đủ thời gian, sử dụng `digitalToggle()` để thay đổi trạng thái LED và cập nhật lại mốc thời gian.

6.3 Bài tập mở rộng

Xây dựng chương trình điều khiển đồng thời ba LED tích hợp với các khoảng đảo trạng thái độc lập:

- `LED_RED` : `500 ms` .
- `LED_BLUE` : `1000 ms` .
- `LED_GREEN` : `1500 ms` .

Không sử dụng `delay()` trong vòng lặp chính. Mỗi LED cần sử dụng một mốc thời gian riêng để quá trình định thời của một LED không làm dừng việc xử lý các LED còn lại.

KẾT QUẢ MONG ĐỢI

Ba LED nhấp nháy đồng thời theo các khoảng thời gian tương ứng và hoạt động độc lập về mặt định thời.

7 Tài liệu tham khảo

[1] Arduino, *Digital Pins* , Arduino Documentation.

[2] Arduino, *Arduino Language Reference* , Arduino Documentation.

[3] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet* , S32K1XX, Rev. 15, 2026.

[4] MaaZEDU, *MaaZEDU Development Board Guide* .